

NanoBlimp

Lab Guide

Format: These modules are designed for laptop demonstration and discussion. Students should run or show their work on their own laptops rather than submitting files.

Bonus boxes are optional. If you are still working through the main task, skip them and come back later.

Module	Topic	Tools
0	Setup & orientation — images are just arrays	Python, NumPy, Matplotlib
1	NumPy fundamentals — shapes, slicing, masking	NumPy
2	Square detection — HSV masks, morphology, contours	OpenCV
3	Live webcam — real-time square/object detection	OpenCV
4	ArUco tags — identify objects in the real world	OpenCV + aruco

Two tracks through this guide:

- | | |
|--------------------------|---|
| New to coding | Follow the “New to coding? Start here” boxes closely. They give you starter code and explain each step in plain English. |
| Experienced coder | Read the documentation tables first, then attempt the task from scratch. Use the starter skeletons only if you get stuck. |

Both tracks end at the same place. The difference is how much scaffolding you use.

Module 0 — Setup & Orientation

Images are just arrays of numbers

Prerequisites — do this before anything else

What you need installed:

- Python 3.9 or newer (python --version to check)
- A terminal / command prompt (search “Terminal” on Mac/Linux, “Command Prompt” on Windows)
- The three libraries below

Open your terminal and run:

```
pip install opencv-python numpy matplotlib
```

If you see **Successfully installed** for all three, you are ready. If you see an error, ask for help before continuing.

Why this module? Before writing a single line of OpenCV code, you need one key idea: a digital image is nothing more than a NumPy array. Every pixel is a number. Once that clicks, everything else in this course makes sense.

What are these libraries?

- **OpenCV** — a library that lets Python work with images and cameras. Think of it as a toolbox for everything visual.
- **NumPy** — how Python stores images as grids of numbers. An image is just a big table of pixel values.
- **Matplotlib** — how we display images and charts inside Python. Like a built-in screen for your data.

You do not need to understand them fully right now. You will build that understanding step by step.

Step 1 — Read the documentation first

Before writing any code, look up each function in the official documentation. You do not need to memorise anything — just understand what each one does and what it returns.

Function	What to read about
cv2.imread()	How to load an image file. What does it return?
cv2.imshow()	How to display an image in a window.
cv2.cvtColor()	What does colour conversion mean? What is BGR?
plt.imshow()	How is this different from cv2.imshow? When would you use each?
np.zeros()	How to create an array of zeros. What arguments does it take?

Step 2 — Hello image

Open a Python file called `module0.py` (or a notebook). Save any image from your computer in the same folder and use that as your test image.

What you should do: Load an image from your computer, convert it from BGR to RGB, display it with Matplotlib, print its **shape**, and print the value of one pixel. Use the output to identify height, width, number of channels, and pixel intensity range.

Starter skeleton — fill in the TODO lines:

```
import cv2
import matplotlib.pyplot as plt

# TODO: change "your_image.jpg" to your actual filename
img = cv2.imread("your_image.jpg")

# TODO: convert from BGR to RGB
img_rgb = cv2.cvtColor(img, _____)

# TODO: display with plt.imshow and plt.show()

# TODO: print img.shape
# TODO: print the value of pixel at row 10, column 10: img[10, 10]
```

Write your observation answer as a **# comment** directly below the relevant line of code.

Observation question (answer as a **# comment** in your code):

Why does the image look wrong (colours are off) if you display it with `plt.imshow(img)` directly, without converting to RGB first?

Common errors in Module 0

NoneType error	<code>cv2.imread()</code> returned nothing. Check that the filename is correct and the image is in the same folder as your script.
Wrong colours	You forgot to convert BGR to RGB before calling <code>plt.imshow()</code> . OpenCV loads images in BGR order; Matplotlib expects RGB.
ModuleNotFoundError	The library is not installed. Run <code>pip install opencv-python numpy matplotlib</code> again in your terminal.

Module 1 — NumPy Fundamentals

Arrays, shapes, slicing, and masking

Why this module? Every camera frame you will ever process is a NumPy array. If you know how to slice, index, and mask arrays, you already know how to manipulate images. These three exercises build that muscle — no camera needed yet.

2.1 Exercise 1A — Shape, slicing, and dtype

A colour image is stored as a 3-D array with shape (**height**, **width**, 3). The three values at each pixel are the Blue, Green, and Red intensities (0–255). In this exercise you will create one from scratch.

Read the docs first

Function	What to read about
<code>np.zeros(shape, dtype)</code>	How to make an array of zeros. What dtype should pixel data use?
<code>np.uint8</code>	What range of values does uint8 hold? Why is it used for images?
<code>array[r1:r2, c1:c2]</code>	NumPy slicing — how to select a rectangular region of an array.
<code>array.shape / .dtype</code>	How to inspect the dimensions and data type of an array.

What you should do: Create a blank $100 \times 100 \times 3$ image using NumPy. Use slicing to draw a red square in one region and a thin green horizontal stripe across the middle. Display the final array as an image.

Starter skeleton:

```
import numpy as np
import matplotlib.pyplot as plt

# Create a blank black image: 100 rows, 100 cols, 3 colour channels
img = np.zeros((100, 100, 3), dtype=np.uint8)

# Draw a red square in the top-left region
# In BGR: red = [0, 0, 255]   green = [0, 255, 0]   blue = [255, 0, 0]
img[10:40, 10:40] = [0, 0, 255]   # TODO: change region or colour

# TODO: draw a thin green horizontal stripe across the middle row
# Hint: rows 49:52 span the middle; all columns =[:, :]

plt.imshow(img[:, :, ::-1])   # flip BGR -> RGB for display
plt.axis("off")
plt.show()
```

Observation question (answer as a `#` comment in your code):

What happens if you create the array with `dtype=np.float64` instead of `np.uint8` and try to display it? Why?

2.2 Exercise 1B — Boolean masking

A mask is an array of True/False values the same shape as your data. You use it to select pixels that meet a condition — for example, “all pixels brighter than 200”. This is one of the most common operations in image processing.

Read the docs first

Function	What to read about
<code>array > value</code>	Comparison on arrays — what does it return?
<code>np.sum(mask)</code>	How to count how many True values are in a boolean array.
<code>array[mask]</code>	Fancy indexing — how to use a boolean mask to select elements.
<code>np.random.randint()</code>	How to generate random integer arrays.

What you should do: Create or load an array, make a boolean mask for values greater than 200, count how many pixels satisfy the condition, then make another mask for values equal to 255. Compare both counts and explain why they may differ.

Starter skeleton:

```
import numpy as np

# Create a random array of pixel-like values (integers 0 to 255)
arr = np.random.randint(0, 256, size=(50, 50), dtype=np.uint8)

# TODO: create a boolean mask for values > 200
mask_200 = _____

# TODO: count how many True values are in the mask
count_200 = np.sum(_____)
print("Pixels > 200:", count_200)

# TODO: create a mask for values == 255 and count them
```

Observation question (answer as a `#` comment in your code):

Could you use this same masking technique to isolate all pixels of a particular colour in an image? How would you extend it to a 2-D array?

2.3 Exercise 1C — Channel splitting

A colour image has three layers (channels) stacked together — Blue, Green, Red. You can pull each one apart using NumPy slicing. This is exactly what you will do every time you filter by colour.

Read the docs first

Function	What to read about
<code>img[:, :, 0]</code>	What does this slice give you? Which channel is index 0 in OpenCV?
<code>np.mean()</code>	How to compute the average value of an array.
<code>np.argmax()</code>	How to find the index of the largest value in an array.

What you should do: Load `square.png`, split the image into its B, G, and R channels using NumPy slicing, display each channel separately, compute the mean of each channel, and identify which channel has the highest average value.

Starter skeleton:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("square.png")    # loads as BGR

# Split into channels using NumPy slicing
blue = img[:, :, 0]
green = img[:, :, 1]
red = img[:, :, 2]

# TODO: display each channel with plt.imshow(..., cmap="gray")
# TODO: compute np.mean() for each channel and print the results
# TODO: which channel has the highest mean?
```

Observation question (answer as a `#` comment in your code):

Look at the square image (`square.png`) — which channel do you expect to be strongest for a yellow square, a blue square, and a pink square? Run your code and check.

Bonus challenge (optional): Load the square image. Create a figure with 4 subplots: the original image, and one per channel. Label each. Which channel best separates one colour of square from the background? Why is one channel alone not enough for all colours?

Module 2 — Contours: Detect and Measure Shapes

From pixels to geometry

In Module 1 you learned to slice and mask NumPy arrays. Now we use that skill on real images — but instead of picking rows and columns by number, we pick pixels by *colour*. A colour mask is just a boolean array that is **True** wherever a pixel falls inside a colour range. Everything else is the same NumPy logic you already know.

Why this module? A contour is the boundary of a shape. Once you have contours, you can measure every detected object — its area, perimeter, size, position, and approximate geometry. This is the foundation of the square-counting task.

3.1 Exercise 2A — Contours on a clean image

We start with a clean, synthetic image — white squares, circles, and triangles on a black background. No heavy preprocessing is needed. This lets you focus entirely on understanding contours before applying the same ideas to the noisy square image.

Read the docs first

Function	What to read about
<code>cv2.findContours()</code>	What inputs does it need? What does it return? What are <code>RETR_EXTERNAL</code> and <code>CHAIN_APPROX_SIMPLE</code> ?
<code>cv2.drawContours()</code>	How to draw contours back onto an image. What does the thickness argument do?
<code>cv2.approxPolyDP()</code>	What is polygon approximation? What does the epsilon parameter control?
<code>cv2.arcLength()</code>	What does it measure? What does the second argument <code>closed=True</code> mean?

What you should do: Create or use a clean test image containing squares, circles, and triangles. Find contours, approximate each contour using `approxPolyDP`, keep only 4-corner convex shapes, and draw outlines only around the square-like objects.

Starter skeleton:

```
import cv2
import numpy as np

# Create a clean test image
canvas = np.zeros((400, 400), dtype=np.uint8)
cv2.rectangle(canvas, (30, 30), (120, 120), 255, -1) # square
cv2.circle(canvas, (250, 80), 60, 255, -1) # circle
cv2.rectangle(canvas, (50, 250), (150, 350), 255, -1) # another square

# Find contours
contours, _ = cv2.findContours(canvas,
                               cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```

output = cv2.cvtColor(canvas, cv2.COLOR_GRAY2BGR)

for cnt in contours:
    # Approximate the contour to a polygon
    peri    = cv2.arcLength(cnt, True)
    approx  = cv2.approxPolyDP(cnt, 0.02 * peri, True)

    # TODO: keep only shapes with 4 corners that are convex
    # TODO: draw a green outline around accepted shapes
    print("Corners:", len(approx))

```

Observation question (answer as a `# comment` in your code):

Why did the circle not get detected as a square? What value did `approxPolyDP` return for its corner count? Write a line to print it out and check.

3.2 Exercise 2B — The noisy square image: why do we preprocess?

Now apply the same idea to `square.png`. This image is harder because it contains many colourful dots, splashes, black holes inside shapes, tilted squares, and non-square shapes. The goal is to understand why preprocessing is needed before contour detection.

Read the docs first

Function	What to read about
<code>cv2.cvtColor(COLOR_BGR2HSV)</code>	Why convert from BGR to HSV before masking colourful shapes?
<code>cv2.inRange()</code>	How to create a binary mask from HSV lower and upper bounds.
<code>cv2.morphologyEx()</code>	What are morphological opening and closing? Why should opening usually come before closing in this noisy image?
<code>cv2.minAreaRect()</code>	How to get a rotated bounding box for tilted square-like shapes.

What is HSV and why use it?

Imagine you want to find all “red” pixels in an image. In BGR, a dark red and a bright red look completely different in all three numbers. In HSV:

- **H (Hue)** — the colour itself (0–179 in OpenCV). Red is near 0 or 179, green near 60, blue near 120.
- **S (Saturation)** — how vivid/colourful (0 = grey, 255 = pure colour).
- **V (Value)** — how bright (0 = black, 255 = full brightness).

To keep all *colourful* pixels (regardless of which colour), allow all hues (0–179), require enough saturation (≥ 50), and enough brightness (≥ 50). This is the range `[0, 50, 50]` to `[179, 255, 255]`.

What you should do: Apply the square-detection pipeline to `square.png`: convert to HSV, create a colour mask, remove tiny dots with opening, fill small gaps with closing, find contours, and draw rotated boxes around contours that pass the square-like filters.

Starter skeleton:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("square.png")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Keep all colourful pixels
lower = np.array([0, 50, 50])
upper = np.array([179, 255, 255])
mask = cv2.inRange(hsv, lower, upper)

# Clean the mask
kernel_open = np.ones((3, 3), np.uint8)
kernel_close = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel_open, iterations=1)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel_close, iterations=2)

# TODO: find contours on the cleaned mask
# TODO: for each contour, compute approxPolyDP, minAreaRect,
#       aspect ratio, and extent; draw a box if it passes all filters
```

Observation question (answer as a # comment in your code):

In Exercise 2A the image was clean. In 2B the image has colourful background dots and black texture inside shapes. Why can closing fuse nearby dots into a larger blob? Why is opening-before-closing usually safer here?

3.3 Exercise 2C — Extract features per detected square candidate

For each detected square candidate compute: Height, Width, Area, Perimeter, Aspect Ratio, and Extent. Store them in a list of dictionaries. These features explain why a contour was accepted or rejected as square-like.

Formula reference

$$\begin{aligned} \text{Area} &= \text{cv2.contourArea}(\text{cnt}) \\ \text{Perimeter} &= \text{cv2.arcLength}(\text{cnt}, \text{True}) \\ \text{Aspect Ratio} &= \frac{\max(w, h)}{\min(w, h)} \\ \text{Extent} &= \frac{\text{Contour Area}}{\text{Rotated Box Area}} \end{aligned}$$

A perfect square has aspect ratio close to 1. Extent tells how much of the rotated bounding box

is actually filled by the contour.

What you should do: For every detected square candidate, compute width, height, area, perimeter, aspect ratio, and extent. Store these values in a list of dictionaries, print the list, and use the values to justify why each contour was accepted or rejected.

Observation question (answer as a `# comment` in your code):

Look at the aspect ratio and extent values. Which false detections passed because their rotated bounding box looked square-like even though the actual contour was messy?

Bonus challenge (optional): Draw both the actual contour and the rotated bounding box for each detected candidate. Which one shows the true messy boundary more clearly?

Module 3 — Live Webcam

Real-time processing on your laptop

A video is just a sequence of images played fast enough to look smooth (typically 30 per second). Each frame is a NumPy array — exactly what you have been working with in Modules 1 and 2. The only new concept is reading frames in a loop from your camera instead of from a file.

Why this module? When the hardware arrives, you will replace `cap.read()` with an HTTP request to the ESP32 camera — everything else stays identical. The code you write here is the foundation.

4.1 Exercise 3A — Build `get_snapshot()`

Read the docs first

Function	What to read about
<code>cv2.VideoCapture(0)</code>	What does the argument 0 mean? When would you use 1 or 2?
<code>cap.read()</code>	What two values does it return? What does the first one tell you?
<code>cv2.resize()</code>	How to change the dimensions of an image. What is the size argument format?
<code>cap.release()</code>	Why is this important? What happens if you skip it?
<code>cv2.waitKey(1)</code>	Why use 1 ms instead of 0 in a video loop?

What you should do: Write a `get_snapshot()` function or loop that opens the webcam, reads frames continuously, displays the live feed, and exits cleanly when the user presses **Q**. Print the frame shape at least once.

Starter skeleton:

```
import cv2

cap = cv2.VideoCapture(0)    # 0 = built-in webcam

while True:
    ret, frame = cap.read()  # ret=True if frame was captured
    if not ret:
        print("Could not read frame. Check your webcam.")
        break

    # TODO: print frame.shape the first time through the loop

    cv2.imshow("Webcam", frame)

    # Press Q to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
break
```

```
cap.release()
cv2.destroyAllWindows()
```

No webcam? Replace the `cap.read()` line with loading a saved image inside the loop — the rest of the code is identical:

```
frame = cv2.imread("your_image.jpg")
ret    = frame is not None
```

Common errors in Module 3:

- | | |
|---------------------------|---|
| Window freezes | You forgot <code>cv2.waitKey(1)</code> inside the loop. This is required for OpenCV to process window events. |
| Black screen | Webcam index is wrong. Try <code>cv2.VideoCapture(1)</code> or (2) if your built-in camera is not index 0. |
| Window won't close | Press Q while the OpenCV window is in focus, not the terminal. |

Observation question (answer as a `# comment` in your code):

What is the shape of the array returned by `cap.read()`? Print it inside the loop to confirm. Is it always the same?

4.2 Exercise 3B — Live square/object detection

Combine the webcam loop with the HSV masking and contour logic from Module 2. For a live demo, hold up a coloured square card or any solid-coloured object and watch the bounding box follow it in real time.

What you should do: Combine live webcam frames with the HSV mask, morphology, contour detection, and square/object filtering pipeline from Module 2. Draw a bounding box around detected objects and update the count every frame.

Observation question (answer as a `# comment` in your code):

Does the bounding box jitter between frames even when the object is still? What might cause this and how could you reduce it?

Bonus challenge (optional): Add a second colour range. Detect two different square/object groups simultaneously — draw boxes in different colours and print both counts separately.

Module 4 — ArUco Tags (Capstone)

Let the camera know what it is looking at

An ArUco tag is a printed square with a unique black-and-white pattern. A camera can detect it instantly and read its ID number — like a barcode designed for computer vision. Robots use them to identify objects, navigate spaces, and measure distances. This is where everything you have learned connects to the real world.

5.1 Exercise 4A — Generate and detect a tag in a saved image

Read the docs first

Function	What to read about
<code>cv2.aruco.getPredefinedDictionary()</code>	What is a dictionary in ArUco? What is <code>DICT_4X4_50</code> ?
<code>cv2.aruco.generateImageMarker()</code>	How to generate a tag image from an ID number.
<code>cv2.aruco.ArucoDetector()</code>	How to create a detector object.
<code>detector.detectMarkers()</code>	What three things does it return: corners, ids, rejected?

What you should do: Generate an ArUco marker image, save it, take or load a photo containing the marker, run the ArUco detector, draw the detected marker boundary, and print the detected ID number.

Starter skeleton — generating a tag:

```
import cv2

# Load a predefined dictionary of tag patterns
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)

# Generate tag with ID = 5 at 200x200 pixels
tag_img = cv2.aruco.generateImageMarker(aruco_dict, 5, 200)
cv2.imwrite("tag_id5.png", tag_img)
print("Tag saved as tag_id5.png")
```

Can't print the tag? Display `tag_id5.png` fullscreen on a phone or second monitor — the detector still works perfectly.

Detecting the tag:

```
img = cv2.imread("tag_id5.png") # or any photo with the tag in it
detector = cv2.aruco.ArucoDetector(aruco_dict,
                                   cv2.aruco.DetectorParameters())

corners, ids, _ = detector.detectMarkers(img)

if ids is not None:
    print("Detected IDs:", ids.flatten())
    cv2.aruco.drawDetectedMarkers(img, corners, ids)
```

```
cv2.imshow("Detected", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Observation question (answer as a `# comment` in your code):

What happens when you tilt or partially cover the tag? At what angle does detection fail?

5.2 Exercise 4B — Live ArUco detection on webcam

What you should do: Run ArUco detection on webcam frames in a loop. Detect visible markers, draw their boundaries on the live feed, and test how detection changes when the marker is moved, tilted, or partially covered.

5.3 Exercise 4C — Give each tag a meaning (capstone)

Assign a label to each tag ID. When the camera sees a tag, display its label on the frame. This is how a robot knows it is looking at “Object A” versus “Object B”.

What you should do: Create a dictionary that maps ArUco tag IDs to object names. During live detection, read each detected ID, look up its label, and display that label above the marker on the video frame.

Hint — mapping IDs to labels:

```
# Define what each tag ID means
tag_labels = {
    0: "Landing Pad",
    1: "Obstacle A",
    7: "Target Zone",
}

# Inside your detection loop:
if ids is not None:
    for tag_id in ids.flatten():
        label = tag_labels.get(tag_id, f"Unknown ID {tag_id}")
        print("Seeing:", label)
        # TODO: draw the label on the frame using cv2.putText()
```

Observation question (answer as a `# comment` in your code):

You have now built a camera that can identify labelled objects in real time. How would you extend this to use the ESP32? What is the only function you would need to change?

Bonus challenge (optional): Generate and print 3 different tags (IDs 0, 1, 7). Place them all in frame at once. Can the detector read all three simultaneously? Add a counter showing how many unique tags are currently visible.

Appendix — Computer Vision Function Reference

Quick reference for the image-processing pipeline

Why this module? This section collects the functions used in the colourful square-detection pipeline. Use it as a teaching reference: students can first read what each function does, then connect it to the exact stage where it appears in the code.

A. Loading and displaying images

Function	What to read about
<code>cv2.imread(path)</code>	Loads an image from a file path. Example: <code>img = cv2.imread("square.png")</code> . OpenCV loads colour images in BGR order.
<code>cv2.cvtColor(img, code)</code>	Converts an image from one colour space to another. Example: <code>cv2.COLOR_BGR2RGB</code> for display with Matplotlib, or <code>cv2.COLOR_BGR2HSV</code> for colour masking.
<code>plt.figure(figsize=(w,h))</code>	Creates a new Matplotlib figure. Example: <code>figsize=(14,8)</code> makes a wide image display.
<code>plt.imshow(image)</code>	Displays an image. Use <code>cmap="gray"</code> for single-channel masks or edge images.
<code>plt.title(text)</code>	Adds a title above the displayed image. Useful when showing pipeline stages one by one.
<code>plt.axis("off")</code>	Hides axis ticks and pixel coordinates.
<code>plt.show()</code>	Displays the current Matplotlib figure.

B. NumPy helpers

Function	What to read about
<code>np.array(list)</code>	Creates a NumPy array. Used for HSV lower and upper bounds, for example <code>np.array([0,50,50])</code> .
<code>np.ones(shape, dtype)</code>	Creates an array filled with ones. Used to create morphology kernels such as <code>np.ones((3,3), np.uint8)</code> .
<code>np.uint8</code>	Unsigned 8-bit integer type. Pixel values usually use this type because image values are commonly in the range 0–255.
<code>np.int32(array)</code>	Converts values to 32-bit integers. Used when converting box corner coordinates before drawing them.
<code>array.copy()</code>	Creates a separate copy of an image. Useful before drawing contours or boxes so the original image remains unchanged.

C. Colour masking

Function	What to read about
<code>cv2.inRange(src, lower, upper)</code>	Creates a binary mask. Pixels inside the given range become 255 (white); pixels outside become 0 (black). Example: keep colourful pixels using HSV lower <code>[0,50,50]</code> and upper <code>[179,255,255]</code> .

Why HSV here? In RGB/BGR, a red square, blue square, and yellow square all need different channel rules. In HSV, we can keep all colourful pixels using saturation and brightness: allow all hues, require enough saturation, and require enough value.

D. Cleaning the mask

Function	What to read about
<code>cv2.morphologyEx(src, op, kernel, iterations=n)</code>	Applies a morphology operation to a binary mask. The <code>op</code> chooses the operation, <code>kernel</code> controls the neighbourhood size, and <code>iterations</code> controls how many times it runs.
<code>cv2.MORPH_CLOSE</code>	Closing operation: dilation followed by erosion. It fills small black holes and connects small gaps inside white shapes.
<code>cv2.MORPH_OPEN</code>	Opening operation: erosion followed by dilation. It removes tiny white noise such as small colourful dots in the background.

Kernel choice: For this noisy square image, remove dots first with a gentle opening kernel such as 3x3. Then fill holes using a smaller closing kernel such as 5x5. A strong 7x7 closing kernel can fuse nearby dots with shapes before they are removed.

E. Optional edge operations

Function	What to read about
<code>cv2.GaussianBlur(src, ksize, sigmaX)</code>	Smooths an image to reduce noise before edge detection. Example: <code>cv2.GaussianBlur(gray, (5,5), 0)</code> .
<code>cv2.Canny(image, threshold1, threshold2)</code>	Detects edges. It outputs a black-and-white edge image where strong boundaries are white. Lower thresholds detect more edges; higher thresholds detect fewer edges.
<code>cv2.dilate(src, kernel, iterations=n)</code>	Expands white regions. After Canny, dilation can thicken thin edge boundaries.

When not to start with Canny: In a noisy image with many colourful splashes, Canny detects too many tiny background edges. For the square image, HSV masking is a cleaner first step than edge detection.

F. Finding and drawing contours

Function	What to read about
<code>cv2.findContours(image, mode, method)</code>	Finds outlines of white connected regions in a binary mask. It returns a list of contours and hierarchy information.
<code>cv2.RETR_EXTERNAL</code>	Contour retrieval mode that keeps only outermost contours. Useful when we care about whole objects and not holes inside them.
<code>cv2.CHAIN_APPROX_SIMPLE</code>	Contour storage method that compresses straight boundary segments and stores fewer points.
<code>cv2.drawContours(img, contours, index, color, thickness)</code>	Draws contours on an image. Use <code>index=-1</code> to draw all contours. The <code>thickness</code> controls line width.

G. Measuring and filtering shapes

Function	What to read about
<code>cv2.contourArea(cnt)</code>	Computes the area inside a contour. Useful for removing tiny dots using a rule such as <code>if area < 150: continue</code> . Area alone does not prove that a shape is a square.
<code>cv2.arcLength(cnt, closed)</code>	Computes the perimeter of a contour. Use <code>closed=True</code> for closed shapes like squares, circles, and triangles.
<code>cv2.approxPolyDP(cnt, epsilon, closed)</code>	Simplifies a contour into a polygon. The <code>epsilon</code> value controls simplification strength. Smaller epsilon keeps more detail; larger epsilon gives fewer corners.
<code>cv2.isContourConvex(cnt)</code>	Checks whether a contour has no inward dents. A normal square is convex; many irregular blobs are not.
<code>cv2.minAreaRect(cnt)</code>	Returns the smallest rotated rectangle that encloses a contour. This works even when the square is tilted. It returns center, size, and angle.
<code>cv2.boxPoints(rect)</code>	Converts a rotated rectangle from <code>minAreaRect</code> into four corner points so it can be drawn.

H. Square-filtering logic used in the pipeline

A detected object is treated as square-like only if it passes several tests:

1. **Area test:** remove very small objects, e.g. `area < 150`.
2. **Corner test:** after `approxPolyDP`, keep only polygons with 4 corners.
3. **Convexity test:** remove shapes with inward dents.
4. **Aspect ratio test:** width and height from `minAreaRect` should be close, e.g. `aspect_ratio <= 1.25`.
5. **Extent test:** contour area should fill enough of its rotated bounding box, e.g. `extent >= 0.45`.

Important teaching point: The white box drawn by `minAreaRect` is a clean geometric approximation. It does not mean the original object is a mathematically perfect square. To show the difference, draw both the actual contour and the rotated bounding box.